

PORTADA

Análisis y Diseño de Sistemas II

Semana 6 — Diseño de Arquitecturas N-Capas

Universidad de Antioquía · Ingeniería de Sistemas



ENCUADRE

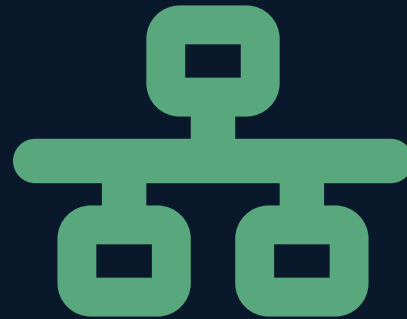
De ocho estilos a uno, en profundidad

La semana pasada vieron ocho estilos arquitectónicos, incluido **cliente/servidor** en sus variantes de 2 y 3 capas. Hoy profundizamos exactamente en eso: cómo se diseña bien una arquitectura de **N capas**, el estilo que la mayoría de los equipos va a usar en su proyecto de Fábrica Escuela.

APERTURA

7 capas.

Más de 40 años sin romperse.



CASO · FICHA TÉCNICA

El modelo OSI, ISO 1984

Nombre	Open Systems Interconnection (OSI)
Organismo	International Organization for Standardization (ISO)
Año de estandarización	1984
Objetivo	Estandarizar cómo se comunican sistemas de cómputo distintos, dividiendo el problema en niveles independientes
Estructura	7 capas apiladas, cada una con una responsabilidad propia
Vigencia	Sigue siendo la referencia conceptual estándar para diseñar redes, más de cuarenta años después

Estándar documentado por ISO/IEC 7498; es, probablemente, el ejemplo más citado de arquitectura en capas de toda la industria.

CASO · POR QUÉ FUNCIONÓ EL DISEÑO EN CAPAS

Cada capa evoluciona sin tocar a las demás

La razón por la que el modelo OSI se sostiene décadas después no es que sus siete capas sean "las correctas" — es que **cada capa solo depende de la capa inmediatamente adyacente**, nunca de los detalles internos de las demás.

- La capa física pasó de cobre a fibra óptica y a redes inalámbricas **sin que las capas superiores tuvieran que cambiar**.
- La capa de aplicación ganó protocolos nuevos (HTTP/2, HTTP/3) sin rediseñar el transporte o la red debajo.

Aislar el cambio en cada nivel es lo que permitió que tecnologías completamente distintas evolucionaran en paralelo, durante décadas, sin romper el sistema completo.

TRANSICIÓN

El mismo principio, aplicado al software de aplicación

OSI organiza *redes* en capas. Pero el mismo principio de aislamiento por niveles —cada capa solo conoce a su vecina inmediata — se aplica también a la **arquitectura interna de una aplicación** de negocio: capas de presentación, lógica de negocio y persistencia.

Hoy formalizamos ese principio con nombre propio: el patrón *Layers*, y cómo diseñarlo bien en un sistema N-capas.

CONCEPTO

Layers: organizar el sistema en niveles

El patrón **Layers** (Buschmann et al., *Pattern-Oriented Software Architecture*) organiza el sistema en **niveles jerárquicos**. Cada capa:

- Ofrece un conjunto de **servicios** a la capa inmediatamente superior.
- Consume los servicios que le ofrece la capa inmediatamente inferior.
- No conoce ni depende de capas que no sean sus vecinas directas.

Es exactamente lo que ilustra OSI: la capa de transporte usa servicios de red, y ofrece servicios a la capa de sesión — nunca "salta" capas.

CONCEPTO

El principio de aislamiento de cambios

La razón de ser del patrón es el **principio de aislamiento de cambios**: si una capa cambia su implementación interna sin cambiar la interfaz que expone, ninguna otra capa debería enterarse.

En una aplicación empresarial típica esto se traduce en tres capas:

- **Presentación** — interfaz de usuario o API expuesta al exterior.
- **Lógica de negocio** — reglas y procesos propios del dominio, las mismas clases que ya modelaron en la Unidad 2.
- **Persistencia** — acceso a la base de datos u otro almacenamiento.

Cambiar el motor de base de datos no debería obligar a tocar la capa de presentación.

CONCEPTO

2, 3 y N capas: no es solo cuestión de contar

Variante	Estructura	Cuándo se usa
2 capas	Cliente habla directo con la base de datos	Sistemas muy pequeños, prototipos, herramientas internas
3 capas	Presentación → Lógica de negocio → Persistencia	El estándar de la mayoría de aplicaciones empresariales, incluida Fábrica Escuela
N capas	Se agregan niveles adicionales: capa de API/gateway, capa de caché, capa de integración con servicios externos	Sistemas más grandes con necesidades específicas por nivel

Agregar una capa no es gratis: cada nivel nuevo debe justificarse con una responsabilidad propia — no se agregan capas "porque suena más profesional".

CONTRAEJEMPLO

El atajo que rompe el aislamiento

Un error frecuente en proyectos de estudiantes: la capa de presentación (un controlador, una vista) accede **directamente** a la capa de persistencia, saltándose la lógica de negocio "para ahorrar una llamada".

Ese atajo destruye la razón de ser del patrón: la validación de negocio queda dispersa o duplicada, y un cambio en la base de datos ahora sí obliga a tocar la presentación — exactamente lo que Layers existe para evitar.

CONCEPTO

Capas estrictas vs. capas relajadas

	Estricta (<i>strict layering</i>)	Relajada (<i>relaxed layering</i>)
Regla	Una capa solo puede llamar a la inmediatamente inferior	Una capa puede llamar a cualquier capa inferior, no solo la vecina
Ventaja	Aislamiento máximo, más fácil de razonar y de sustituir una capa completa	Evita "capas de paso" que solo reenvían la llamada sin agregar valor
Riesgo	Puede generar capas vacías que solo delegan hacia abajo	Erosiona el aislamiento — es más fácil terminar en el contraejemplo anterior

Para el proyecto del curso: empiecen con capas estrictas — es más fácil de mantener disciplinado en equipos pequeños y con poco tiempo de desarrollo.

CONCEPTO · VENTAJAS Y LIMITACIONES

Lo que Layers gana — y lo que cuesta

Ventajas	Limitaciones
Bajo acoplamiento entre capas: cada una se puede reemplazar o probar de forma aislada	Cada capa añade una llamada adicional: puede introducir overhead de rendimiento frente a acceder directamente al nivel inferior
Facilita el mantenimiento: un cambio queda contenido en su capa	Las capas a veces se "filtran" (<i>leaky abstractions</i>): un detalle de la capa inferior termina siendo visible en la superior
Facilita las pruebas: se puede simular (mockear) la capa vecina	Diseñar demasiadas capas "por si acaso" agrega complejidad sin beneficio real



CONCEPTO · APLICADO A UN FRAMEWORK REAL

Controller — Service — Repository

El mapeo más común de las tres capas a código, en frameworks como Spring Boot o ASP.NET:

Capa	Componente típico	Responsabilidad
Presentación	Controller	Recibe la solicitud HTTP, valida el formato de entrada, delega al servicio
Lógica de negocio	Service	Aplica las reglas de negocio, orquesta operaciones sobre varias entidades
Persistencia	Repository	Traduce operaciones a consultas sobre la base de datos

El **Controller** nunca debería construir una consulta SQL, y el **Repository** nunca debería contener una regla de negocio.

TENSIÓN CONCEPTUAL

Aislamiento vs. rendimiento

Cada capa que se atraviesa agrega una llamada, y potencialmente serialización de datos entre niveles. En sistemas con requisitos de rendimiento muy exigentes, ese costo acumulado puede ser significativo.

¿Vale la pena pagar ese costo por el aislamiento que ofrece Layers? Para la gran mayoría de aplicaciones empresariales, sí — el costo es mínimo comparado con el beneficio de mantenibilidad. Para sistemas de muy alto rendimiento (ej. trading de alta frecuencia), a veces se sacrifica aislamiento por velocidad.



INTERACCIÓN

Ubiquen las capas de su propio proyecto

En equipos, en salas de Zoom:

1. Listen las clases de dominio ya modeladas en la Unidad 2 — todas pertenecen a la capa de lógica de negocio.
2. Identifiquen qué tecnología van a usar en cada capa (ej. React o HTML/JS para presentación, Spring Boot para negocio, JPA/Hibernate para persistencia).
3. Señalen un ejemplo de operación de su proyecto que atraviese las tres capas de punta a punta.

10 minutos · resultado insumo directo para el diagrama de componentes de mañana

CONCEPTO EN DISPUTA

¿Tres capas siempre alcanzan?

Postura minimalista: tres capas (presentación, negocio, persistencia) son suficientes para la enorme mayoría de sistemas — agregar más capas sin una razón concreta solo añade indirección innecesaria.

Postura de escala: sistemas más grandes sí se benefician de capas adicionales explícitas — por ejemplo, una capa de API/gateway que unifica la entrada de varios clientes (web, móvil), o una capa de caché entre negocio y persistencia para reducir carga sobre la base de datos.

No hay una respuesta universal: depende de si existe una responsabilidad real que hoy vive mezclada dentro de otra capa y necesita aislarse.

CONCEPTO · PUENTE A LA PRÓXIMA SESIÓN

De las capas al diagrama formal

Hasta ahora las capas se han descrito en tablas y conversación. **Mañana esa misma estructura se formaliza con notación UML estándar:** un diagrama de componentes muestra las capas y sus interfaces; un diagrama de despliegue muestra en qué nodos físicos corre cada una.

El contenido no cambia — cambia, otra vez, la precisión con la que se comunica.

SÍNTESIS

Ideas clave de hoy

1. El patrón **Layers** organiza el sistema en niveles donde cada capa solo conoce a su vecina inmediata — el principio de **aislamiento de cambios**.
2. Las tres capas típicas: **presentación, lógica de negocio y persistencia** — mapeadas en código a `Controller` , `Service` y `Repository` .
3. **Capas estrictas** maximizan el aislamiento; **capas relajadas** evitan capas de paso vacías — para el proyecto, empezar estricto.
4. Cada capa agrega **overhead de rendimiento**; el aislamiento vale ese costo en la enorme mayoría de sistemas empresariales.
5. Agregar capas adicionales (API/gateway, caché) debe responder a una responsabilidad real, no a moda.



CIERRE

Para mañana

Traigan identificadas las **tres capas de su proyecto** con la tecnología concreta que va en cada una — es el insumo directo para construir el diagrama de componentes.

Mañana formalizamos esta estructura con diagramas UML de despliegue, paquetes y componentes.

